Vanessa Beatriz G. Beltrame
24034783-2

1 · JavaScript

```
console      0.1    3      0.3
console      9007199254740993
```

- **(a) Saída:**

- **(b) Detecção do problema: Nunca**.

  - **Explicação:**

    1. Devido à representação em ponto flutuante no padrão IEEE 754 (onde `0.1` é uma dízima em binário), `0.1 * 3` resulta em `0.30000000000000004`, o que torna a comparação com `0.3` falsa.

    2. `9007199254740993` excede o limite de precisão inteira segura para *doubles* em JS ($2^{53} = 9007199254740992$), resultando na perda de precisão e arredondamento silencioso para o par mais próximo (`9007199254740992`). Não há erros de compilação nem exceções em tempo de execução.

2 · Python

```
"maçã"
  len      len
```

- **(a) Saída:**

- **(b) Detecção do problema: Nunca** (não é um problema/erro, é o comportamento esperado).

    - **Explicação:**

        - `len(p)` conta a quantidade de *code points* (caracteres Unicode), retornando **4**.

        - `p.encode()` converte a string para a codificação padrão UTF-8, retornando um objeto de *bytes*. Como os caracteres "ç" e "ã" consomem 2 bytes cada em UTF-8 (enquanto "m" e "a" usam 1 byte cada), a contagem total de bytes é **6** (`1 + 1 + 2 + 2`).

3 · Go

```
var   byte   255
```

- **(a) Saída:**

- **(b) Detecção do problema: Nunca**.

    - **Explicação:** Em Go, o tipo `byte` é um alias para `uint8` (inteiro sem sinal de 8 bits, variando de 0 a 255). Incrementar `255` faz com que o

valor estoure (*overflow*) e retorne a zero em silêncio durante a execução, sem gerar avisos ou pânico do sistema.

4 · Java

```
int       new int 3
                  0
                  3
```

- **(a) Saída:**

(Seguido da interrupção do programa devido à exceção)

- **(b) Detecção do problema:** Na **Execução** (*runtime*).

  - **Explicação:**

    - `v[0]` imprime `0` (valor padrão atribuído a posições de *arrays* numéricos não inicializados em Java).

    - `v[3]` tenta acessar um índice fora do limite do vetor (já que um vetor de tamanho 3 possui índices `0`, `1` e `2`). O Java realiza a verificação de limites dos vetores em tempo de execução e lança a exceção `java.lang.ArrayIndexOutOfBoundsException`.

5 · Rust

```
let     String       "oi"
let
```

```
println! "{} {}"
```

- **(a) Saída:** Não gera saída (falha antes da execução).

- **(b) Detecção do problema:** Na **Compilação**.

    - **Explicação:** Rust utiliza o sistema de posse e transferência de propriedade (*ownership*). Ao atribuir `let t = s;`, a posse do objeto `String` é movida de `s` para `t`. A partir desse momento, a variável `s` torna-se inválida. Tentar usar `s` na macro `println!` resultará em um erro de compilação (`error[E0382]: borrow of moved value: s`).

6 · C

```
union { int    float
      1.0f
printf "%d\n"
```

- **(a) Saída:**

  (Nota: O valor inteiro pode variar dependendo da arquitetura/plataforma, mas em sistemas de 32/64 bits padrão IEEE 754 este é o valor gerado).

- **(b) Detecção do problema: Nunca**.

    - **Explicação:** Em C, uma `union` aloca memória compartilhada para todos os seus membros. Atribuir `1.0f` ao campo `f` escreve a representação binária IEEE 754 do valor `1.0` de ponto flutuante na memória (`0x3F800000`). Ao ler `u.i`, a linguagem simplesmente reinterpreta essa mesma sequência de bits como um número inteiro com sinal, imprimindo

seu equivalente decimal (`1065353216`), sem realizar nenhuma verificação de tipo ou sinalizar erro.